AI Knowledge Engine

Technical & Product Handbook

A retrieval-augmented generation service built with NestJS, pgvector, Cloudflare Workers AI, and Gemini embeddings.

1. Overview

The AI Knowledge Engine is a backend service that lets a team ask plain-English questions and get answers drawn only from its own documents. It is built with NestJS and TypeScript. The project was created as a hands-on way to learn production Retrieval-Augmented Generation, and it doubles as a portfolio piece.

The core promise is simple: the system never invents facts. Every answer is grounded in stored content, and the exact source passages are returned alongside the answer so a reader can audit where it came from.

2. The Problem It Solves

Large language models are confident but forgetful. Ask a raw model about your company policies and it will happily make something up, because it has never seen your data. This behaviour is called hallucination.

Retrieval-Augmented Generation fixes this by fetching the most relevant passages from a trusted knowledge base first, then asking the model to answer using only those passages. The result is an assistant that is accurate, current, and traceable to a source.

3. Architecture at a Glance

Requests arrive at a NestJS HTTP layer organised into feature modules: chat, documents, query, health, and a shared LLM module. PostgreSQL with the pgvector extension stores documents and their vector embeddings; the demo can use a hosted Neon database instead of a local database container. The current demo uses Cloudflare Workers AI for answer generation and Google Gemini for document and query embeddings.

The design keeps ingestion (turning documents into searchable vectors) cleanly separate from querying (answering a question). This separation makes each part easy to test and reason about.

4. Language Model Providers

The engine talks to language models through a single provider interface, so the underlying vendor can be swapped without touching business logic. The supported generation providers are Ollama, Google Gemini, OpenAI, Groq, and Cloudflare Workers AI. The active answer-generation provider is selected with LLM_PROVIDER.

The current demo uses Cloudflare Workers AI with the @cf/meta/llama-3.2-3b-instruct model to generate answers. Cloudflare Workers Free currently includes a limited daily Workers AI allocation of 10,000 Neurons; it is not unlimited free inference. When the free allocation is exhausted, requests can fail. Check the current account quota and keep paid billing disabled if the demo must remain at zero cost.

5. The Embedding Model

An embedding is a list of numbers that captures the meaning of a piece of text. Texts with similar meaning end up close together in this numeric space, which is what makes semantic search possible.

The current demo uses Google Gemini Embedding 2 for both document chunks and search questions, requesting vectors with 768 dimensions to match the pgvector column. Document embeddings and query embeddings must use the same embedding model and compatible settings; vectors from different embedding models cannot be safely mixed. Cloudflare generates answers from the retrieved text and does not participate in vector search.

6. Chunking Strategy

Whole documents are too large to embed usefully, so each document is split into smaller overlapping pieces called chunks. Small chunks embed with sharper meaning, and the overlap prevents an idea from being cut in half at a boundary.

The current chunker uses a sliding window of roughly five hundred characters with about one hundred characters of overlap between neighbours. A future improvement is token-aware sizing, which measures length in model tokens rather than raw characters.

7. Vector Storage and Search

Each chunk and its embedding are stored in Postgres using the pgvector extension, which adds a native vector column type. Similarity between a question and a chunk is measured with cosine distance; a smaller distance means a closer meaning, and the score returned to the caller is one minus that distance.

To keep search fast as data grows, an HNSW index is built over the embedding column using cosine operators. HNSW is an approximate nearest-neighbour index, trading a tiny amount of accuracy for a large gain in speed.

8. Document Ingestion

Documents enter the system in two ways. Raw text can be posted directly as JSON, or a file can be uploaded. File upload accepts plain text and PDF files up to ten megabytes; extracted text is limited to 50,000 characters before chunking. Visitor uploads are isolated to a browser session, each session can store up to three documents, and uploads expire after seven days. Both paths share the same downstream pipeline of chunk, embed, and store.

Ingestion is idempotent. A SHA-256 hash of the normalised text is stored with a unique constraint, so uploading identical content a second time is a no-op: the existing document is returned and nothing is embedded again. Unsupported file types, empty files, and image-only scanned PDFs are rejected with a clear error.

9. The Query Flow (RAG)

Answering a question follows three steps: retrieve, augment, and generate. First the question is embedded and the top matching chunks are retrieved. Then those chunks are inserted into the prompt as numbered sources. Finally the language model writes an answer using only that supplied context.

The response includes the written answer, the list of source chunks with their similarity scores,

a grounded flag, and token usage counts. Returning the sources is what makes the answer auditable rather than a black box.

10. Grounding and Guardrails

The system prompt instructs the model to answer strictly from the provided sources and to say it does not know when the answer is absent. This is the guardrail that stops the assistant from falling back on general world knowledge.

For example, if the knowledge base only contains store policies and someone asks who painted the Mona Lisa, the correct behaviour is to decline, because that fact is not in the documents even though the model happens to know it.

11. Optional Tool Calling

Beyond retrieval, the engine includes a small demonstration of tool or function calling. Two example tools are exposed to the model: one returns the current date and one counts the exact number of words in a piece of text.

When a tool is needed the model requests it, the backend runs the real function, and the result is fed back so the model can finish its answer. A safety limit of five rounds prevents runaway loops. Basic retrieval does not need tools; they matter for more advanced agentic behaviour where the model decides when to act.

12. Data Model

Two main tables hold the knowledge base. The documents table records the title, the full original content, the source type of text, txt, or pdf, the content hash used for deduplication, and an optional visitor-session owner. The document_chunks table holds each chunk, its position within the document, and its 768-dimension Gemini embedding vector.

Chunks reference their parent document and are removed automatically if the parent is deleted, keeping the store consistent.

13. Roadmap and Phases

The project is delivered in phases. Phase one covered the chat foundation, structured output, and the tool-calling demo. Phase two added pgvector storage and semantic search. Phase three delivered grounded question answering and file upload with deduplication.

The current demo includes anonymous browser-session isolation for visitor uploads, an upload limit of three documents per session, and seven-day upload expiry. This is demo isolation, not account-grade authentication. User accounts, durable multi-tenant authentication, async ingestion, response caching, and a full audit trail remain future work.

14. Non-Functional Qualities

Reliability matters as much as features. Provider or configuration errors return a 503 Service Unavailable, invalid model output returns a 502 Bad Gateway, and bad input returns a 400 Bad Request. Requests are validated against typed schemas before any work begins.

This demo sends document text to Gemini for embeddings and sends retrieved text to Cloudflare Workers AI for answer generation. Do not upload confidential or sensitive material. Gemini and Cloudflare quotas, data handling terms, and free allocations are provider-controlled

and may change; check the current terms and usage before a demo.

15. Glossary

Embedding: a numeric vector representing the meaning of text. Chunk: a small overlapping slice of a document. Cosine similarity: a measure of how close two vectors point in the same direction. Grounding: restricting answers to retrieved context. Idempotency: repeating an operation has no additional effect. HNSW: an approximate nearest-neighbour index used for fast vector search.